

Real-Time Data Ingestion and Processing for Word Count Application

Upgraded Project Documentation

Technology: Apache Kafka, PySpark Structured Streaming, Java 17, Python 3.11

Repository: VectorsMap-04 / real-time-data-ingestion-and-processing-for-word-count-application

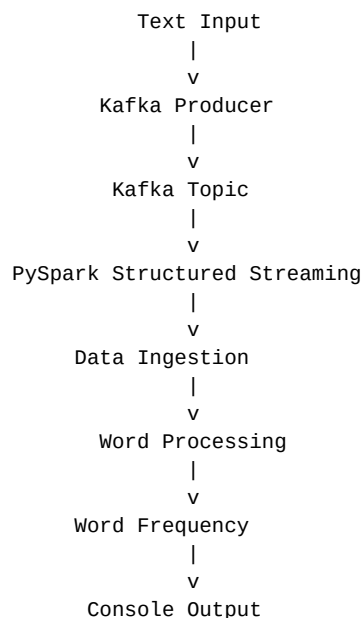
1. Project Overview

This project implements a real-time data ingestion and stream-processing pipeline for a Word Count Application. Text messages are ingested through Apache Kafka and processed continuously using PySpark Structured Streaming. The application extracts incoming message values, tokenizes the text into words, aggregates word frequencies, and displays the processed results in real time.

2. Objective

The main objective is to demonstrate how a real-time data stream can be ingested, transported through a messaging system, processed using a stream-processing engine, and converted into useful analytical output. The project provides a practical implementation of Kafka-based ingestion and Spark Structured Streaming processing in an isolated Linux development environment.

3. System Architecture and Workflow



4. Step-by-Step Workflow

Step 1 – Environment Setup: A project-specific Python virtual environment and local Java 17 runtime are used to keep project dependencies isolated.

Step 2 – Kafka Startup: ZooKeeper and the Kafka broker are started using the project's shell scripts.

Step 3 – Data Ingestion: A Kafka producer sends text messages to the configured Kafka topic.

Step 4 – Stream Consumption: PySpark Structured Streaming connects to Kafka through the Spark SQL Kafka connector and continuously reads the incoming stream.

Step 5 – Text Processing: The message value is extracted and split into individual words.

Step 6 – Aggregation: Words are grouped and their frequencies are calculated from the streaming data.

Step 7 – Output: The processed word-frequency results are written to the console for real-time observation.

5. Technology Stack

Technology	Version / Role
Python	3.11.13 – application environment
Java	17.0.20.1 – Spark runtime
PySpark	3.5.9 – stream processing
Apache Kafka	3.7.2 – real-time ingestion and messaging
Spark SQL Kafka Connector	3.5.9 – Kafka integration
Ubuntu	26.04 LTS – operating environment

6. Project Structure

```
real-time-data-ingestion-and-processing-for-word-count-application/  
|  
+-- app/  
|   +-- realtime_wordcount.py  
|  
+-- jars/  
|   +-- spark-sql-kafka-0-10_2.12-3.5.9.jar  
|  
+-- documentation/  
|   +-- MSME_Original_Project_Documentation.pdf  
|   +-- Upgraded_Project_Documentation.pdf  
|   +-- MSME_Project_Certificate.jpeg  
|  
+-- activate.sh  
+-- requirements.txt  
+-- run-wordcount.sh  
+-- start-kafka.sh  
+-- start-zookeeper.sh  
+-- .gitignore  
+-- README.md
```

7. Environment Isolation

The project uses local project resources and a Python virtual environment so that its software configuration can be managed independently from other development projects. Java 17, Python 3.11, PySpark, Kafka, and the required Spark-Kafka connector are configured for this application.

8. Execution Procedure

Activate the environment:

```
cd ~/real-time-data-ingestion-and-processing-for-word-count-application  
source activate.sh
```

Start ZooKeeper:

```
./start-zookeeper.sh
```

Start Kafka in another terminal:

```
./start-kafka.sh
```

Run the application:

```
./run-wordcount.sh
```

9. Example

Input messages:

```
hello world  
hello kafka  
kafka spark
```

Example word-frequency output:

```
hello    2  
kafka    2  
world    1  
spark    1
```

10. Key Concepts Demonstrated

- Real-time data ingestion
- Event-driven data processing
- Kafka producers, consumers, and topics
- PySpark Structured Streaming
- Stream processing and aggregation
- Word-frequency analysis
- Distributed data processing concepts
- Environment and dependency isolation

11. Documentation and Project Evidence

The GitHub repository preserves the original MSME project documentation alongside this upgraded documentation and the project completion certificate. Keeping both versions makes the evolution of the project traceable while retaining the original project evidence.

12. Future Enhancements

- Window-based word-frequency analysis for time-based streaming insights
- Top-N word detection and ranking
- Persistent storage using PostgreSQL or another database
- Real-time dashboards using Power BI or Grafana
- Multiple Kafka partitions and scalable deployment
- Fault-tolerant and checkpoint-based processing
- Docker-based deployment

- Cloud deployment using managed streaming infrastructure
- Real-time application monitoring and alerting
- REST API for accessing processed results
- Advanced text analytics and Natural Language Processing (NLP)

13. Conclusion

The upgraded project demonstrates a complete real-time data ingestion and processing workflow using Apache Kafka and PySpark Structured Streaming. It provides practical experience with message-based ingestion, continuous stream processing, aggregation, and real-time output. The modular design also provides a foundation for extending the application toward persistent storage, dashboards, scalable deployment, and advanced analytics.

14. Author

Venky

GitHub: <https://github.com/VectorsMap-04>